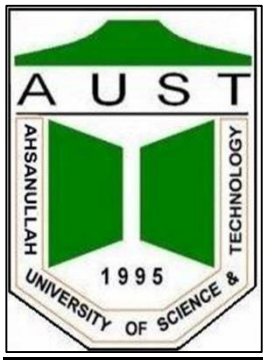
AHSANULLAH UNIVERSITY OF SCIENCE AND TECHNOLOGY



Department of Computer Science and Engineering

Course No: CSE 2104
Course Title: Data Structures Lab

Assignment No: 01

Date of Submission: 15/12/2025

Submitted by,
Name: Durjoy Roy
Student ID: 00724105101055
Lab Section: B1
Academic Semester: Spring 2025
Program: BSc in CSE

Question:

A bookstore maintains a sorted list of unique book IDs to quickly identify whether a book is available in stock. You are required to write a C++ program that uses Binary Search to determine if a specific book ID is present in the list. Additionally, the store often needs to find the first book ID that is not smaller than a given value (lower bound) and the first book ID that is

greater than a given value (upper bound) in order to manage restocking and categorization. Given a sorted array of book IDs and a query book ID, your program should output whether the book exists in the list, the lower bound position and the upper bound position corresponding to the query.

Tasks

1. Read the number of book IDs and then read the sorted list of unique book IDs.
2. Read the query book ID to be searched.
3. Implement Binary Search to check if the book ID exists.
4. Implement logic to find the lower bound of the query.
5. Implement logic to find the upper bound of the query.
6. Display the search result along with the lower and upper bound indices.

You should write user defined functions for Binary Search, Lower Bound and Upper Bound. Input should be taken inside the main function. All user defined functions must be called from the main function.

Sample Input Sample Output

7

10 20 30 40 50 60 70

40

Book Found

Lower Bound Index: 3

Upper Bound Index: 4

6

5 7 7 8 8 11

8

Book Found

Lower Bound Index: 3

Upper Bound Index: 5

Solution:

Code:

```
#include<bits/stdc++.h>
using namespace std;

bool binarySearch(vector<int>arr, int key)
{
   int low = 0, high = arr.size() - 1;

   while (low <= high)
   {
      int mid = low + (high - low) / 2;

      if (arr[mid] == key)
         return true;
      else if (arr[mid] < key)
         low = mid + 1;
      else
         high = mid - 1;
   }
   return false;
}

int lowerBound(vector<int>arr, int key)
{
   int low = 0, high = arr.size() - 1;
   int ans = arr.size();

   while (low <= high)
   {
```

```
        int mid = low + (high - low) / 2;

        if (arr[mid] >= key)
        {
            ans = mid;
            high = mid - 1;
        }
        else
            low = mid + 1;
    }
    return ans;
}

int upperBound(vector<int>arr, int key)
{
    int low = 0, high = arr.size() - 1;
    int ans = arr.size();

    while (low <= high)
    {
        int mid = low + (high - low) / 2;

        if (arr[mid] > key)
        {
            ans = mid;
            high = mid - 1;
        }
        else
            low = mid + 1;
    }
```

```
    return ans;
}

int main()
{
    int n;
    cin >> n;

    vector<int> books(n);
    for (int i = 0; i < n; i++) cin >> books[i];

    int key;
    cin >> key;

    if (binarySearch(books, key))
        cout << "Book Found" << endl;
    else
        cout << "Book Not Found" << endl;

    cout << "Lower Bound Index: " << lowerBound(books, key) << endl;
    cout << "Upper Bound Index: " << upperBound(books, key) << endl;

    return 0;
}
```

Input Ouput Sample:

```
7
10 20 30 40 50 60 70
40
```

Binary Search Dry Run:

- low = 0, high = 6
- mid = 3 → books[mid] = 40 → 40 == 40 → Found!
- Result: Book Found at index 3

Lower Bound Dry Run (first index ≥ key):

- low = 0, high = 6, ans = 7
- mid = 3 → books[mid] = 40 → 40 ≥ 40? Yes → ans = 3, high = mid - 1 = 2
- mid = 1 → books[mid] = 20 → 20 ≥ 40? No → low = mid + 1 = 2
- mid = 2 → books[mid] = 30 → 30 ≥ 40? No → low = mid + 1 = 3
- Result: Lower Bound Index = 3

Upper Bound Dry Run (first index > key):

- low = 0, high = 6, ans = 7
- mid = 3 → books[mid] = 40 → 40 > 40? No → low = mid + 1 = 4
- mid = 5 → books[mid] = 60 → 60 > 40? Yes → ans = 5, high = mid - 1 = 4
- mid = 4 → books[mid] = 50 → 50 > 40? Yes → ans = 4, high = mid - 1 = 3
- Result: Upper Bound Index = 4

Ouput:

Book Found

Lower Bound Index: 3

Upper Bound Index: 4